

Audix ROS 2 Workspace

This workspace runs the Audix Raspberry Pi robot stack. It connects the Pi, ESP32 micro-ROS firmware, GPIO hardware, IR sensors, scissor lift, webcam, YOLO shelf scanner, robot mission manager, and a simple browser dashboard.

The main launcher is `robot_start.sh`. It starts the full robot stack and prints the dashboard URL, normally `http://<pi-ip>:8080`.

What This Workspace Does

Audix is a small warehouse audit robot. The ROS workspace:

- Receives telemetry from the ESP32 drive controller through micro-ROS.
- Sends drive commands to the ESP32 as text commands on a ROS topic.
- Reads six IR proximity sensors from Raspberry Pi GPIO.
- Controls a buzzer and stepper-driven scissor lift through Raspberry Pi GPIO.
- Runs a webcam node and a YOLO shelf scanning service.
- Provides high-level robot services for manual movement, rotation, homing, stop, and shelf audit missions.
- Serves a simple web dashboard that calls ROS services and displays telemetry, IR state, camera frames, mission events, and vision scan results.

Packages

Package	Purpose
<code>audix_interfaces</code>	Custom ROS messages and services used by the robot stack.
<code>audix_robot</code>	Pi-side control nodes, robot manager, GPIO hardware, micro-ROS bridge, and web dashboard.
<code>warehouse_vision</code>	Webcam publisher and YOLO shelf audit service.
<code>firmware/audix_esp32_microros</code>	ESP32 micro-ROS firmware. It subscribes to drive commands and publishes telemetry JSON.

Main Launch Files

File	Purpose
<code>launch/audix_main.launch.py</code>	Full runtime stack used by <code>robot_start.sh</code> : micro-ROS agent, base node, GPIO node, manager, webcam, vision, dashboard.
<code>launch/audix_bridge.launch.py</code>	Smaller bridge-only launch: micro-ROS agent, base node, GPIO node. No manager, vision, or dashboard.

Default namespace is `/audix`, so relative names like `esp/telemetry` appear as `/audix/esp/telemetry`.

Nodes Started By `robot_start.sh`

Node	Package / executable	Main job
<code>/micro_ros_agent</code>	<code>micro_ros_agent</code> <code>micro_ros_agent</code>	Serial micro-ROS transport to ESP32 on <code>/dev/ttyAMA0</code> by default.
<code>/audix/micro_ros_base</code>	<code>audix_robot</code> <code>micro_ros_base_node</code>	Converts ESP32 micro-ROS telemetry JSON into ROS messages and exposes low-level drive services.
<code>/audix/gpio hardware</code>	<code>audix_robot</code> <code>gpio hardware_node</code>	Reads IR sensors and controls buzzer/lift GPIO.
<code>/audix/robot_manager</code>	<code>audix_robot</code> <code>robot_manager_node</code>	High-level command gate, manual safety, homing, map audit mission, lift/vision orchestration.
<code>/audix/webcam</code>	<code>warehouse_vision</code> <code>webcam_node</code>	Publishes camera frames.
<code>/audix/vision_audit</code>	<code>warehouse_vision</code> <code>vision_audit_node</code>	Runs YOLO scan service for shelf/product checking.

Node	Package / executable	Main job
/audix/web_dashboard	audix_robot web_dashboard_node	Serves the simple browser dashboard and HTTP API.

When the ESP32 firmware is connected through the micro-ROS agent, it creates the micro-ROS node `/audix/esp32_controller`. That firmware node subscribes to `/audix/esp/move_goal` and publishes `/audix/esp/telemetry_json`.

Other Executable Nodes

These console scripts are available from `setup.py` but are not launched by `robot_start.sh`.

Executable	Purpose
terminal_move_node	Keyboard/terminal client for direction moves, rotate, stop, reset odom, init IMU.
terminal_teleop_node	Keyboard/terminal RPM teleop using the <code>/audix/twist</code> service.
hello_node	Small test/demo node in <code>warehouse_vision</code> .

There is also an older source file, `audix_robot/esp_uart_bridge_node.py`, for a direct UART JSON bridge. It is not used by `robot_start.sh`; the current stack uses micro-ROS.

Topics

Topic	Type	Publisher	Subscriber	Purpose
/audix/esp/move_goal	std_msgs/String	micro_ros_base_node	ESP32 firmware	Text commands to the ESP32: <code>MOVE</code> , <code>TWIST</code> , <code>STOP</code> , <code>INIT_IMU</code> , <code>RESET_ODOM</code> , etc.
/audix/esp/telemetry_json	std_msgs/String	ESP32 firmware	micro_ros_base_node	Raw JSON telemetry from the ESP32.
/audix/esp/telemetry	audix_interfaces/EspTelemetry	micro_ros_base_node	robot_manager_node, web_dashboard_node	Parsed robot pose, IMU, move, RPM, PWM, encoder telemetry.
/audix/esp/telemetry_raw	std_msgs/String	micro_ros_base_node	Optional debug tools	Raw JSON telemetry republished for debugging.
/audix/ir/state	audix_interfaces/IrState	gpio_hardware_node	robot_manager_node, web_dashboard_node	Six binary IR sensor states and active sensor list.
/audix/mission/event	std_msgs/String	robot_manager_node	web_dashboard_node	Human-readable mission and safety events.

Topic	Type	Publisher	Subscriber	Purpose
<code>/audix/vision/scan_result</code>	<code>std_msgs/String</code> JSON	<code>robot_manager_node</code>	<code>web_dashboard_node</code>	Latest audit scan result as JSON.
<code>/audix/image_raw</code>	<code>sensor_msgs/Image</code>	<code>webcam_node</code>	<code>vision_audit_node</code> , <code>web_dashboard_node</code>	Live camera frames.

Note: the Next.js GUI joystick currently publishes `/cmd_vel` through rosbridge when `NEXT_PUBLIC_ROSBRIDGE_URL` is set, but this ROS workspace does not launch a node that subscribes to `/cmd_vel`. The working control path today is through the manager services or the dashboard HTTP API.

Messages

`audix_interfaces/msg/EspTelemetry`

Parsed ESP32 telemetry:

- Header and `mode`, `seq`.
- IMU fields: `imu_ok`, `yaw_deg`, `raw_yaw_deg`, `gyro_dps`, `gyro_filt_dps`.
- Odom/move fields: `forward_cm`, `strafe_cm`, `progress_cm`, `remaining_cm`, `local_forward_cm`, `local_strafe_cm`, `move_phase`, `move_angle_deg`, `move_distance_cm`, `heading_target_deg`, `heading_error_deg`, `move_done`.
- Wheel fields: `rpm[4]`, `target_rpm[4]`, `pwm[4]`, `raw_encoder_counts[4]`, `signed_encoder_counts[4]`.
- `raw_json` copy of the source telemetry.

`audix_interfaces/msg/IrState`

Six binary IR sensors:

- `front_left`, `front`, `front_right`, `right`, `back`, `left`
- `active`: names of sensors currently detecting an obstacle

Services

All services below are normally under `/audix`.

Service	Type	Server	Purpose
<code>/audix/esp/ping</code>	<code>std_srvs/Trigger</code>	<code>micro_ros_base_node</code>	Check for fresh ESP telemetry.
<code>/audix/esp/init_imu</code>	<code>std_srvs/Trigger</code>	<code>micro_ros_base_node</code>	Publish <code>INIT_IMU</code> to ESP32.
<code>/audix/esp/reset_odom</code>	<code>std_srvs/Trigger</code>	<code>micro_ros_base_node</code>	Reset Pi world odom accumulator and publish <code>RESET_ODOM</code> .
<code>/audix/esp/reset_encoders</code>	<code>std_srvs/Trigger</code>	<code>micro_ros_base_node</code>	Reset Pi odom accumulator and publish <code>RESET_ENC</code> .
<code>/audix/esp/stop</code>	<code>std_srvs/Trigger</code>	<code>micro_ros_base_node</code>	Publish <code>STOP</code> to ESP32.
<code>/audix/move</code>	<code>audix_interfaces/Move</code>	<code>micro_ros_base_node</code>	Low-level move by angle, distance, and heading.
<code>/audix/twist</code>	<code>audix_interfaces/TwistCommand</code>	<code>micro_ros_base_node</code>	Low-level RPM command: forward, strafe, turn.
<code>/audix/esp/raw_command</code>	<code>audix_interfaces/RawCommand</code>	<code>micro_ros_base_node</code>	Send custom ESP32 text command.
<code>/audix/gpio/set_buzzer</code>	<code>std_srvs/SetBool</code>	<code>gpio_hardware_node</code>	Turn buzzer on/off.

Service	Type	Server	Purpose
/audix/lift/move_steps	audix_interfaces/LiftMoveSteps	gpio_hardware_node	Move lift stepper by signed direction and step count.
/audix/manager/set_mode	audix_interfaces/SetRobotMode	robot_manager_node	Set manual, mission, or idle.
/audix/manager/direction_move	audix_interfaces/DirectionCommand	robot_manager_node	High-level safe movement using directions F, B, L, R, FL, FR, BL, BR.
/audix/manager/rotate	audix_interfaces/RotateCommand	robot_manager_node	Rotate left/right by degrees.
/audix/manager/start_audit	audix_interfaces/AuditMission	robot_manager_node	Start map audit mission for selected shelf sides and levels.
/audix/manager/go_home	std_srvs/Trigger	robot_manager_node	Return to odom zero/home.
/audix/manager/stop	std_srvs/Trigger	robot_manager_node	Cancel mission, stop robot, turn buzzer off, return to manual mode.
/audix/scan_shelf	audix_interfaces/ShelfScan	vision_audit_node	Run YOLO shelf scan on a fresh camera frame.

Custom Service Fields

Type	Request	Response
Move	angle_deg, distance_m, heading_deg, timeout_s, wait_for_done	ok, result, message, forward_cm, strafe_cm, heading_deg, raw_json
TwistCommand	forward_rpm, strafe_rpm, turn_rpm, timeout_s	ok, message, raw_json
DirectionCommand	direction, distance_cm, timeout_s	ok, result, message, forward_cm, strafe_cm, heading_deg
RotateCommand	direction, degrees, timeout_s	ok, result, message, heading_deg
SetRobotMode	mode	ok, message, active_mode
LiftMoveSteps	steps, direction, speed_sps	ok, message
AuditMission	shelves, level_1, level_2	accepted, message
ShelfScan	shelf_id	success, shelf_id, expected_product, expected_count, detected_count, detected_products, wrong_products, confidence, status, message, image_path
RawCommand	command, timeout_s, wait_for_done	ok, type, result, message, raw_json

Simple Web Dashboard

The simple GUI is served by web_dashboard_node, not by the Next.js app. Open:

```
http://<pi-ip>:8080
```

The dashboard shows:

- Current mode, last event, telemetry age, odom forward/strafe, yaw, IMU state, move state, and sequence number.
- Manual jog controls for 8 directions.
- Rotate left/right controls.
- STOP, home, init IMU, reset odom, buzzer, and lift controls.
- IR sensor indicators.
- Mission audit side/level selection.
- Live camera JPEG refreshed from the ROS image topic.
- Manual shelf scan and latest audit scan result/image.
- Log of HTTP API responses and mission events.

Dashboard HTTP API

The browser calls these routes on the same Pi/dashboard host:

Route	Method	Uses ROS service/topic
/ or /index.html	GET	Serves the dashboard HTML.
/api/status	GET	Returns cached /audix/esp/telemetry, /audix/ir/state, /audix/mission/event, latest scan.
/api/camera.jpg	GET	Returns latest JPEG encoded from /audix/image_raw.
/api/audit_image? path=...	GET	Returns annotated scan image from ~/audix/audit_images.
/api/move	POST	Calls /audix/manager/direction_move. Body: direction, distance_cm, optional timeout_s.
/api/rotate	POST	Calls /audix/manager/rotate. Body: direction, degrees, optional timeout_s.
/api/mode	POST	Calls /audix/manager/set_mode. Body: mode.
/api/audit	POST	Calls /audix/manager/start_audit. Body: shelves, level_1, level_2.
/api/stop	POST	Calls /audix/manager/stop.
/api/home	POST	Calls /audix/manager/go_home.
/api/init_imu	POST	Calls /audix/esp/init_imu.
/api/reset_odom	POST	Calls /audix/esp/reset_odom.
/api/buzzer	POST	Calls /audix/gpio/set_buzzer. Body: on.
/api/lift	POST	Calls /audix/lift/move_steps. Body: steps, optional direction, optional speed_sps.
/api/scan	POST	Calls /audix/scan_shelf. Body: shelf_id.

This HTTP API is the easiest bridge for an advanced GUI: keep the ROS stack running on the Pi and have the new GUI call these endpoints.

Next.js GUI In This Repo

There is also a separate advanced-looking Next.js GUI in:

```
../Audix_GUI/Audix_GUI
```

It currently uses `lib/mock-data.ts` for most robot, inventory, IR, health, and log data. The joystick component can publish to rosbridge if `NEXT_PUBLIC_ROSBRIDGE_URL` is set, but it publishes `geometry_msgs/Twist` to `/cmd_vel`, and the current ROS stack does not subscribe to that topic.

To migrate this GUI to live robot data, use one of these paths:

1. Easiest path: call the existing dashboard HTTP API on port `8080`.
 - Read status from `GET http://<pi-ip>:8080/api/status`.
 - Show camera from `http://<pi-ip>:8080/api/camera.jpg`.

- Move with `POST /api/move`.
- Rotate with `POST /api/rotate`.
- Stop with `POST /api/stop`.
- Start audit with `POST /api/audit`.
- Scan shelves with `POST /api/scan`.

2. More ROS-native path: run `rosbridge_server` and connect with `roslibjs`.

- Subscribe to `/audix/esp/telemetry`, `/audix/ir/state`, `/audix/mission/event`, `/audix/vision/scan_result`, and `/audix/image_raw` or a compressed camera topic.
- Call `/audix/manager/direction_move`, `/audix/manager/rotate`, `/audix/manager/stop`, `/audix/manager/start_audit`, `/audix/scan_shelf`, and the other services directly.
- If you want joystick `/cmd_vel`, add a ROS node that subscribes to `/cmd_vel` and converts it into `/audix/twist` service calls, or change the GUI to call `/audix/twist` through rosbridge services.

3. Backend path: build a small Node/Python server beside the GUI.

- The backend talks to ROS using `rclpy`, `rclnodejs`, or `roslibjs`.
- The frontend talks to that backend over REST/WebSocket.
- This is useful if the advanced GUI needs authentication, logging, or custom data shaping.

How Data Moves Through The Robot

Command path:

```
Browser dashboard or advanced GUI
-> dashboard HTTP API or ROS service
-> /audix/manager/* service
-> /audix/move or /audix/twist service
-> /audix/esp/move_goal topic
-> ESP32 micro-ROS firmware
-> motors / IMU / encoders
```

Telemetry path:

```
ESP32 firmware
-> /audix/esp/telemetry_json
-> micro_ros_base_node
-> /audix/esp/telemetry
-> robot_manager_node and web_dashboard_node
-> /api/status or advanced GUI ROS subscription
```

GPIO/safety path:

```
Pi GPIO IR sensors
-> gpio_hardware_node
-> /audix/ir/state
-> robot_manager_node safety logic
-> /audix/esp/stop and buzzer when needed
```

Vision path:

```
webcam_node
-> /audix/image_raw
-> vision_audit_node
-> /audix/scan_shelf service
-> robot_manager_node publishes /audix/vision/scan_result
-> web_dashboard_node and GUI
```

Running The Robot Stack

On the Raspberry Pi:

```
cd ~/path/to/ros2_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
./robot_start.sh
```

Then open the URL printed by the script:

```
http://<pi-ip>:8080
```

The script automatically sources:

```
/opt/ros/jazzy/setup.bash
install/setup.bash
```

It also checks that `audix_robot`, `audix_interfaces`, and `micro_ros_agent` are available before launching.

Useful `robot_start.sh` Environment Variables

Set these before running `./robot_start.sh` when needed:

Variable	Default	Purpose
AUDIX_PORT	/dev/ttyAMA0	ESP32 micro-ROS serial port.
AUDIX_BAUD	115200	ESP32 serial baud rate.
AUDIX_DASHBOARD_PORT	8080	Web dashboard HTTP port.
AUDIX MOCK_IR	false	Use mock IR sensor state.
AUDIX MOCK_GPIO	false	Use mock GPIO outputs/lift/buzzer.
AUDIX_CAMERA_ENABLED	true	Start or skip webcam node.
AUDIX_CAMERA_INDEX	0	OpenCV camera index.
AUDIX_VISION_ENABLED	true	Start or skip YOLO vision node.
AUDIX_VISION_CONFIDENCE	0.5	YOLO confidence threshold.
AUDIX_VISION_TARGET_COUNT	2	Expected product count per shelf scan.
AUDIX_VISION_SCAN_SETTLE	0.5	Delay before scan capture during audit.
AUDIX_SIDE_1_LEVEL_1_SHELF_ID	beans_can	Product/shelf id for audit side 1 level 1.
AUDIX_SIDE_1_LEVEL_2_SHELF_ID	indomie	Product/shelf id for audit side 1 level 2.
AUDIX_SIDE_2_LEVEL_1_SHELF_ID	indomie	Product/shelf id for audit side 2 level 1.
AUDIX_SIDE_2_LEVEL_2_SHELF_ID	fruit_rings_cereal	Product/shelf id for audit side 2 level 2.

Variable	Default	Purpose
AUDIX_CLEAN_START	true	Stop old Audix processes before launching.

Example mock run without hardware:

```
AUDIX MOCK_IR=true AUDIX MOCK_GPIO=true AUDIX_CAMERA_ENABLED=false AUDIX_VISION_ENABLED=false  
./robot_start.sh
```

Example different dashboard port:

```
AUDIX_DASHBOARD_PORT=8090 ./robot_start.sh
```

Running Individual Tools

After building and sourcing:

```
source /opt/ros/jazzy/setup.bash  
source install/setup.bash
```

Terminal direction move UI:

```
ros2 run audix_robot terminal_move_node
```

Terminal RPM teleop:

```
ros2 run audix_robot terminal_teleop_node
```

Call a service manually:

```
ros2 service call /audix/manager/direction_move audix_interfaces/srv/DirectionCommand "{direction: F,  
distance_cm: 20.0, timeout_s: 10.0}"
```

Watch telemetry:

```
ros2 topic echo /audix/esp/telemetry
```

Watch IR sensors:

```
ros2 topic echo /audix/ir/state
```

Running The Next.js GUI

The separate Next.js GUI is not launched by `robot_start.sh`.


```
cd ../Audix_GUI/Audix_GUI
npm install
npm run dev
```

It runs on:

```
http://localhost:4000
```

To connect it to the real Pi using the current easiest path, replace mock data calls with fetches to:

```
http://<pi-ip>:8080/api/status
http://<pi-ip>:8080/api/camera.jpg
http://<pi-ip>:8080/api/move
http://<pi-ip>:8080/api/rotate
http://<pi-ip>:8080/api/stop
http://<pi-ip>:8080/api/audit
http://<pi-ip>:8080/api/scan
```

If the GUI is served from a different host/port, the dashboard API may need CORS headers added in `web_dashboard_node.py`, or the Next.js app can proxy requests through its own API routes.

Quick Migration Checklist For An Advanced GUI

- Replace `lib/mock-data.ts` with live data from `/api/status` or ROS topics.
- Use `/api/camera.jpg` for the live camera preview, or add a proper MJPEG/WebRTC stream later.
- Map control buttons to `/api/move`, `/api/rotate`, `/api/stop`, `/api/home`, `/api/init_imu`, `/api/reset_odom`, `/api/buzzer`, and `/api/lift`.
- Map audit controls to `/api/audit` and shelf scan controls to `/api/scan`.
- For direct ROS integration, subscribe to `/audix/esp/telemetry`, `/audix/ir/state`, `/audix/mission/event`, `/audix/vision/scan_result`, and call the `/audix/manager/*` services.
- Do not rely on `/cmd_vel` unless you add a ROS node that converts `/cmd_vel` to the current `/audix/twist` or `/audix/manager/direction_move` control path.